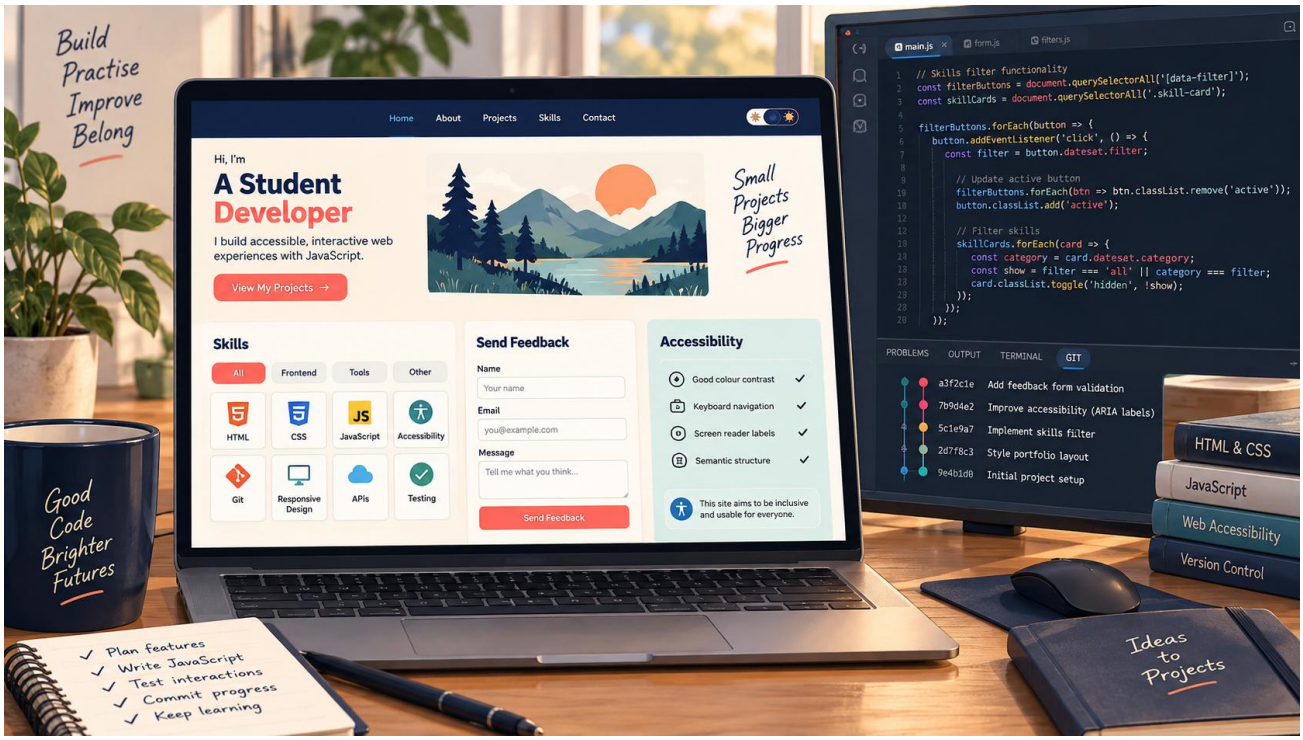
LAB 2 · JAVASCRIPT

Interactive Portfolio Enhancements

Forms · Data-Driven UI · Personalization

How your work will be assessed

You will extend your Lab 1 portfolio with working JavaScript features. Assessment considers the final behavior, the quality of your code, your Git history, your short report and your ability to explain the implementation live.



Illustrative example only — build the features within your own Lab 1 portfolio and visual identity.

Read Section 01 first, then complete the full cumulative track for your target grade.

SECTION 01

Start here

A quick map of the lab, the cumulative grade tracks and the evidence you need to collect while you work.

Contents

01	Start here	2
02	Common requirements	3
03	Git process evidence	4
04	Report evidence	5
05	Grade 3 track	6
06	Grade 4 track	8
07	Grade 5 track	10
08	Technical reference	13
09	Seminar & code defense	14
10	FAQ & resources	15
11	Submission checklist	16

Choose your target grade

Lab 2 is cumulative: Grade 4 includes the complete Grade 3 feature, and Grade 5 includes the complete Grade 3 and Grade 4 features.

Target	Complete feature set	Minimum total commits	Main JavaScript focus
Grade 3	Newsletter & feedback form	10	Events, validation and DOM feedback
Grade 4	Grade 3 + data-driven skills showcase	25	Arrays, rendering, filtering and sorting
Grade 5	Grades 3 & 4 + accessibility display panel	45	Settings, persistence and accessible state

The short version

Update your Lab 1 portfolio, work individually, commit meaningful changes as you build, submit the repository and report, and be ready to demonstrate the highest complete track that works reliably.

SECTION 02

Common requirements

These expectations apply to every grade track. Higher tracks add features; they do not replace earlier requirements.

What everyone must submit

- Your updated Lab 1 portfolio files: HTML, CSS, JavaScript, images and any other local assets needed to open the site.
- A link to your Git repository with the complete commit history intact.
- A .gitignore file and a README.md that states your target grade, describes the implemented features and explains how to run the site.
- A short report in Word or PDF with the required screenshots, explanations and reflection.
- A live seminar demonstration of the feature set required for your target grade.

Working rules

- Complete the lab individually. You may discuss approaches with classmates, but the submitted code and Git history must represent your own work.
- Use vanilla JavaScript only—no jQuery or framework-specific JavaScript. A CSS framework may be used for styling.
- Commit throughout development and across more than one work session. The commit total is a minimum, not a reason to split one change into many meaningless commits.
- Test all features at mobile and desktop widths and resolve errors shown in the browser console.
- Keep code readable: use meaningful names, small focused functions and comments that explain important reasoning.

Browser-only project

Lab 2 has no backend. Form data is not emailed or sent to a server. The Grade 5 feature may save display settings locally in the browser or encode them in the page URL, depending on the method you choose.

How cumulative grading works

- Grade 3: complete the feedback form feature and all common requirements.
- Grade 4: complete Grade 3, then add the full skills showcase.
- Grade 5: complete Grades 3 and 4, then add the full accessibility display panel.

Choose reliability over a broken higher track

Submit the highest complete track you can demonstrate confidently. A fully working lower track is stronger evidence than several unfinished features.

SECTION 03

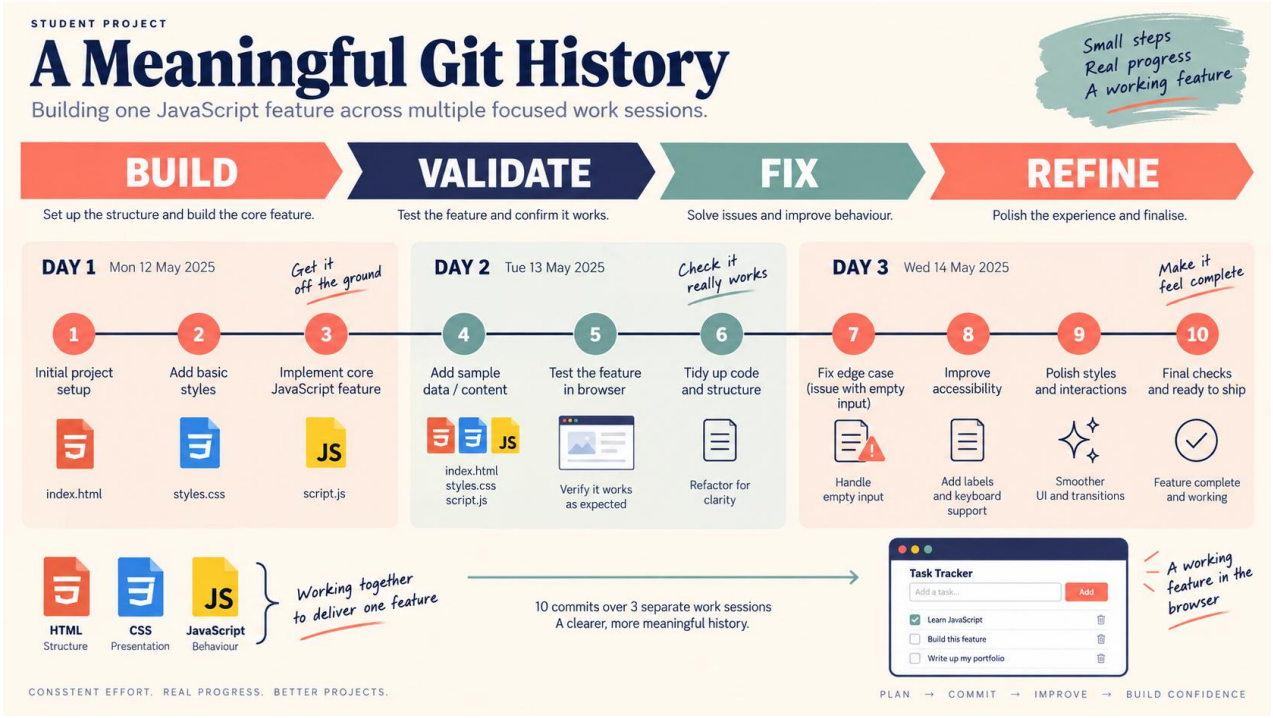
Git process evidence

Your history should show how the features developed—not a group of last-minute commits created only to reach a number.

Target grade	Grade 3 work	Grade 4 addition	Grade 5 addition	Minimum total
Grade 3	10 commits	—	—	10
Grade 4	10 commits	+15 commits	—	25
Grade 5	10 commits	+15 commits	+20 commits	45

A useful commit history

- Commit after a meaningful unit of work: structure, behavior, testing, a fix or a focused refinement.
- Write messages that describe the change, such as “Add email validation” or “Fix active filter after sorting.”
- Keep genuine intermediate states. A real history may include an attempt that you later revised.
- Do not rewrite timestamps or manufacture commits after the work is finished.



Good process evidence connects small, meaningful commits to visible progress over several sessions.

Avoid vague messages

Messages such as “update,” “stuff” or “final” do not explain what changed. Commit messages should help another reader understand the development story.

SECTION 04

Report evidence

Keep the report concise and specific. It should help the reader verify what works and understand one or two important decisions—not repeat this assignment brief.

Required report structure

1. Project information: your name, student ID, target grade, repository link and a short list of implemented features.
2. Feature evidence: include the screenshots required for your target grade and give each screenshot a short caption.
3. Code explanation: identify the main functions for each feature and explain how an event leads to a visible page update.
4. Problem solving: describe at least one real problem, what you tried and the solution that worked.
5. Reflection: answer the three prompts below in your own voice.

Screenshot evidence by track

Feature	Required view	What the caption should identify
Grade 3 form	Normal, error and valid/success states	The input used and the validation response
Grade 4 skills	A filter and a sort applied together	The active filter, sort choice and updated counter
Grade 5 display	Default view and large-text/high-contrast view	The active settings and how they were tested

Reflection prompts

- Which interaction are you most confident explaining, and why?
- How does your Git history show the way your solution changed during development?
- If you had one more development session, what would you improve or test further?

Screenshot quality

Crop screenshots only to remove irrelevant empty space. Keep controls, messages and browser state readable, and check that no private tabs, accounts or personal information are visible.

SECTION 05

Grade 3 track

Add a clear, accessible feedback form that validates input and gives immediate, friendly feedback in the page.

GRADE 3	Feedback form · validation · live counter · clear DOM feedback · 10 total commits

Form structure

Create feedback.html, or place the feature in a clearly labelled section of an existing portfolio page.

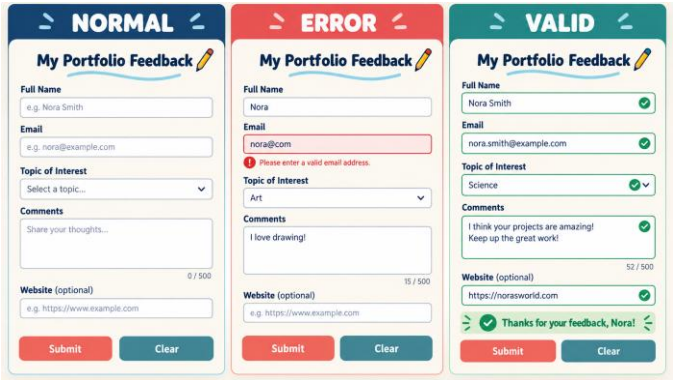
Field	HTML type	Validation and guidance
Full Name	text	Required; at least 2 characters. Allow international letters, spaces, apostrophes and hyphens.
Email	email	Required; must contain a plausible address with @ and a domain.
Topic of Interest	select	Required; include a placeholder plus at least 3 meaningful choices.
Comments	textarea	Required; minimum 15 characters and connected to the live counter.
Website	url	Optional; when filled, use the browser's URL validation.

HTML and accessibility

- Connect a visible label to every form control.
- Include a Submit button and a Clear button with the correct button types.
- Place each error message directly beside or below its field and associate it with the input.
- Provide a status area for the success message so it can be announced by assistive technology.

Required visual states

- Normal: neutral border before the user interacts.
- Focus: a strong visible outline or border for keyboard and pointer users.
- Error: red border plus a written error message—do not rely on color alone.
- Valid: green border only after the field has been checked and passed validation.
- Use smooth, restrained transitions and a responsive layout that works on mobile and desktop.



GRADE 3 BEHAVIOR

JavaScript requirements

Organize the code

- Select each required form element once and store the reference in a const. Do not repeatedly query the same element inside event handlers.
- Use focused functions such as validateFullName(), validateEmail(), validateTopic(), validateComments(), showFieldError(), clearFieldError() and resetForm().
- Keep validation rules and DOM updates separate enough that each function has one clear responsibility.

Live character counter

- Update the count on every input event and state the 15-character minimum in text.
- Use red while the response is too short and green after the minimum is met, while keeping the written count understandable without color.

Submission behavior

1. Listen for submit and call event.preventDefault() so the page does not reload.
2. Run every validation function. Show the relevant messages and stop when any required field is invalid.
3. When all fields are valid, show “Thanks for your feedback, [Full Name]!” in the status area.
4. Reset field values, validation classes, error messages and the counter after a successful submission.
5. Remove the success message automatically after 4 seconds. The Clear button must also restore the complete initial state.

No email is sent

The success message confirms that the browser-side interaction worked. Do not promise that an inbox was updated, because Lab 2 does not include a server or email service.

Grade 3 evidence and checkpoint

- Repository: at least 10 meaningful commits in total.
- Report: form screenshots showing normal, error and valid/success states; explain the validation flow and one problem you solved.
- Seminar: demonstrate one invalid submission, one successful submission and the live counter.

Grade 3 completion check

Working feedback form + accessible validation states + 10 meaningful commits + common report, repository and seminar requirements.

SECTION 06

Grade 4 track

Complete Grade 3, then turn a JavaScript data array into an interactive skills showcase that can be filtered and sorted.

GRADE 4	Includes Grade 3 · 8+ skills · filter + sort · dynamic rendering · 25 total commits

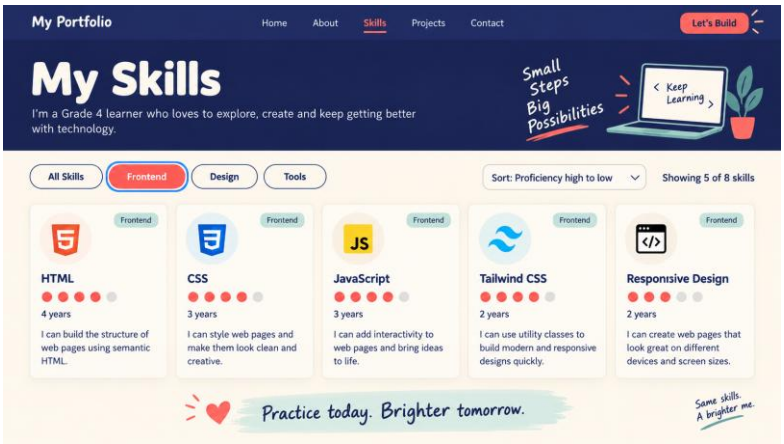
Skill data

Create a JavaScript array containing at least 8 skill objects across at least 3 categories. Every object must use the same fields.

Property	Type	Requirement
id	number	Unique value for each skill.
name	string	Clear skill name, such as Responsive Layouts.
category	string	One of at least 3 consistent categories.
proficiency	number	Integer from 1 to 5.
yearsExperience	number	Zero or more; decimals such as 1.5 are allowed.
description	string	One or two sentences explaining how you use the skill.

Dynamic skill cards

- When the page loads, call one rendering function that loops through the current data and creates one .skill-card per skill.
- Each card shows the name, category badge, visual proficiency indicator, years of experience and description.
- Do not write eight finished cards manually in the HTML; the array must be the source of the displayed content.



Grade 4 interface example: the active filter, sort choice and number of visible results should remain clear.

GRADE 4 BEHAVIOR

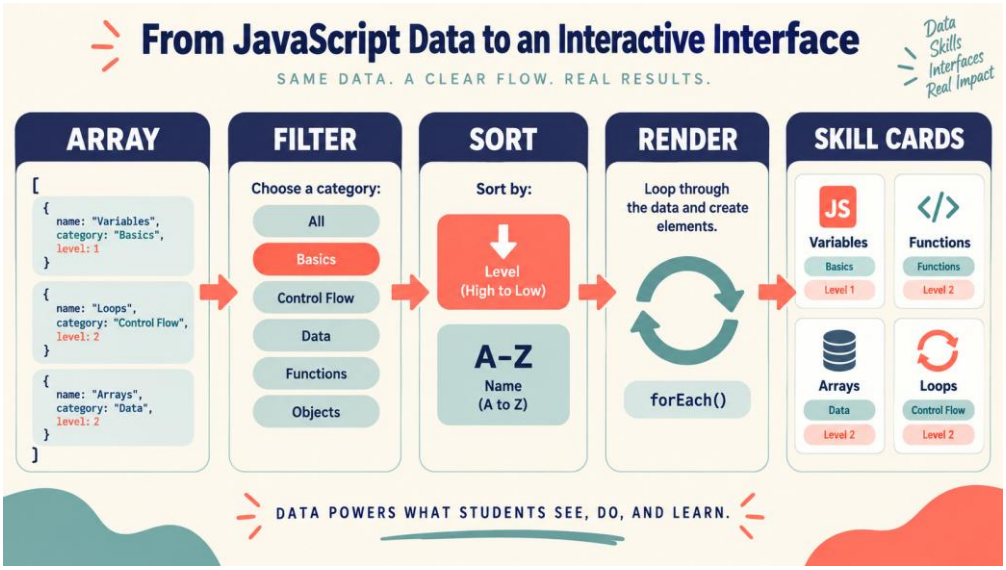
Filtering, sorting and feedback

Filter controls

- Provide an All Skills button and one button for each category. Store the category in a data-category attribute.
- Clicking a filter updates the visible cards and gives the active button a clear visual state.
- Use native buttons, visible keyboard focus and aria-pressed so the active choice is available without relying on color alone.

Sort and combined state

- Sort by proficiency (high to low), years of experience (high to low) and name (A–Z).
- Filtering and sorting must work together. Apply the current filter first, sort the resulting copy, then render it.
- Update text such as “Showing 5 of 8 skills” after every filter change.
- Keep transitions smooth and restrained; respect reduced-motion preferences if you animate filtering or reordering.



The original array is the source; the active filter and sort produce the collection that is rendered.

Grade 4 evidence and checkpoint

- Repository: at least 15 additional meaningful commits after Grade 3—25 commits in total.
- Report: show a filter and sort applied together; explain the data structure, render function and one state-management problem.
- Seminar: change the filter and sort order, then trace how the array becomes the visible cards.

Grade 4 completion check

Complete Grade 3 + data-driven skills showcase + combined filtering/sorting + 25 total commits + all common evidence requirements.

SECTION 07

Grade 5 track

Complete Grades 3 and 4, then add an accessible display panel that changes text size and contrast and restores the visitor's choice.

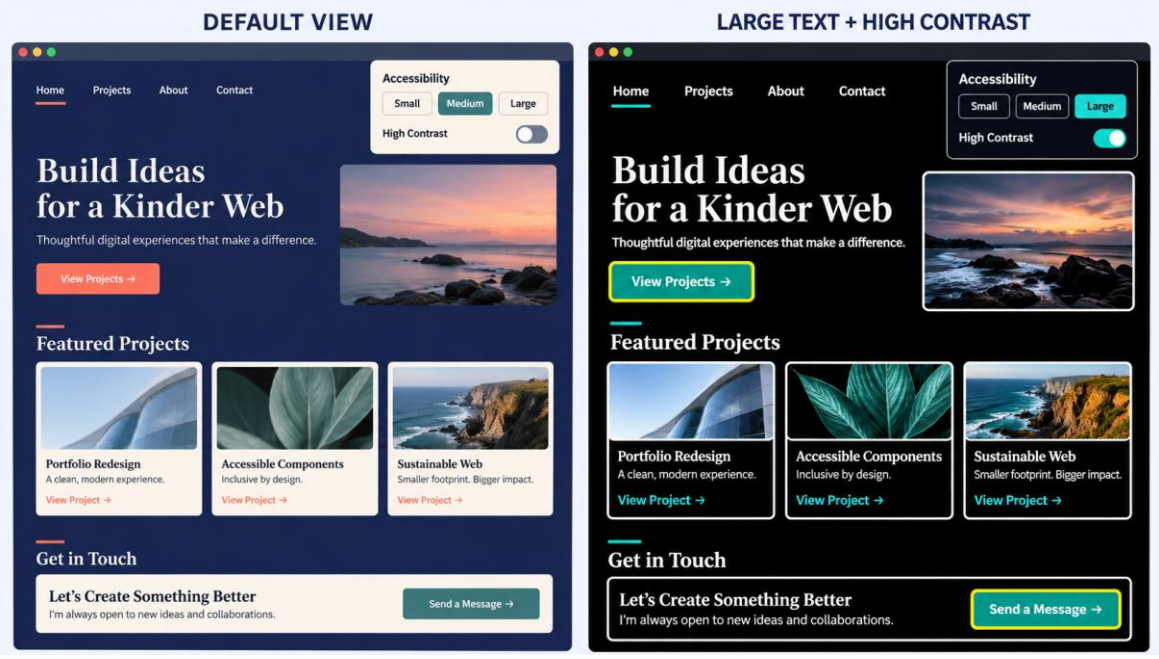
GRADE 5	Includes Grades 3 & 4 · display settings · persistence · keyboard access · 45 total commits

Panel controls

- Provide Small, Medium (default) and Large text-size buttons. Use data-textsize values to identify the choices.
- Provide a Normal/High Contrast toggle. Keep its current state visible and expose it with aria-pressed.
- Group the controls under a clear heading or legend and ensure every control works with Tab and Enter or Space.

Applying settings

- Use one applySettings() function to remove previous setting classes, apply the new classes and update active-control styling.
- Set a clear base size for each option—for example 14px, 16px and 20px—and verify that layout, navigation, forms and cards still work.
- High-contrast mode needs strong foreground/background contrast and clear borders, links, focus rings, errors and success states.
- Use a short transition of about 0.3 seconds, and disable nonessential motion when reduced motion is requested.



Grade 5 interface example: every combination of size and contrast must remain readable and operable.

GRADE 5 BEHAVIOR

Remembering the visitor’s choice

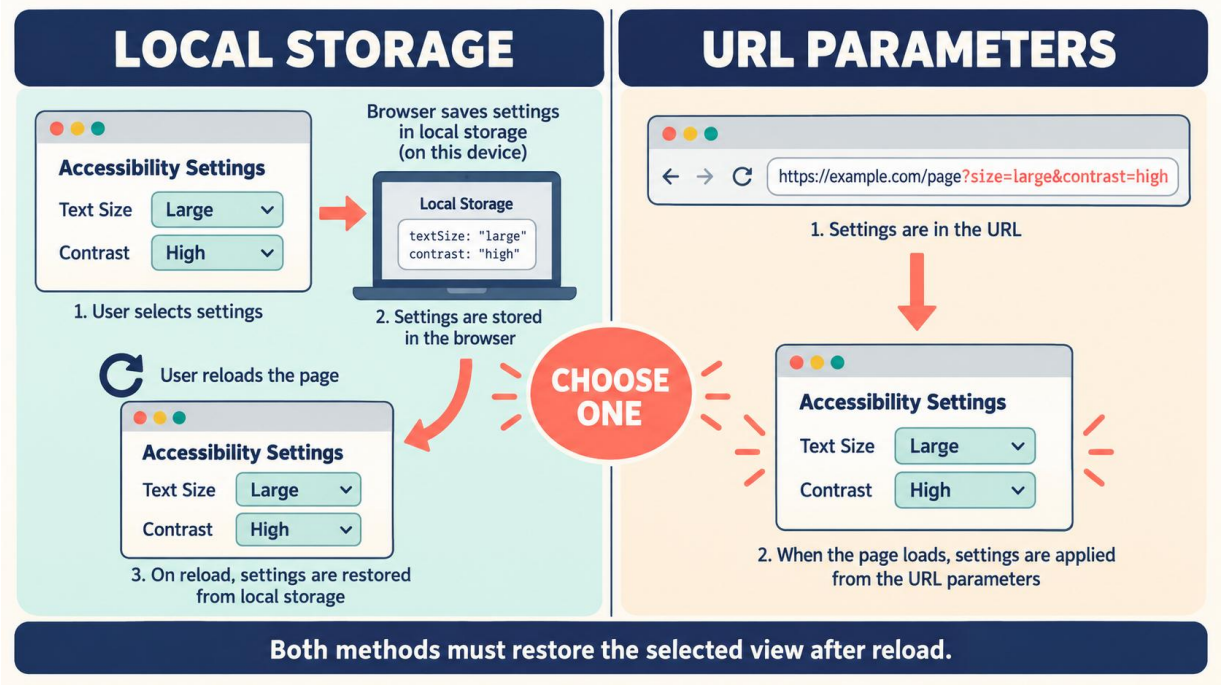
Implement one method below. Both methods must restore the selected text size and contrast after a page reload.

Method 1 · localStorage

- Save textSize and contrast whenever either value changes.
- On page load, read the saved values and fall back to medium text and normal contrast when nothing is stored.
- Apply settings as early as practical so the page does not flash in the wrong style.

Method 2 · URL parameters

- Write both settings into the URL, for example ?size=large&contrast=high.
- Read the parameters on page load, validate the values and fall back to the defaults when a value is missing or invalid.
- If the setting should continue across several portfolio pages, preserve the parameters in internal links.



Choose one persistence method and be ready to explain its trade-offs during the seminar.

GRADE 5 VERIFICATION

Testing and completion evidence

Accessibility test

Text size	Normal contrast	High contrast
Small	Test	Test
Medium (default)	Test	Test
Large	Test	Test

- Test all six combinations: three text sizes in normal contrast and three in high contrast.
- Use the keyboard only to reach and activate every panel control.
- Reload the page and confirm that the selected view returns without a console error.

Grade 5 evidence and checkpoint

- Repository: at least 20 additional meaningful commits after Grade 4—45 commits in total.
- Report: compare the default view with large text + high contrast; explain applySettings(), your persistence method and one accessibility test.
- Seminar: change a setting, reload the page and show that the choice is restored.

Grade 5 completion check

Complete Grades 3 and 4 + accessible persistent display panel + 45 total commits + all common evidence requirements.

SECTION 08

Technical reference

Use this as a final code-quality and file-organization check. It supports the grade sections; it does not add a fourth feature.

Code quality for all tracks

- Use const by default and let only when a variable must be reassigned. Do not use var.
- Choose meaningful names such as validateEmail() rather than check1().
- Make each function responsible for one clear task.
- Use comments to explain important reasoning, edge cases or trade-offs—not to translate every line into English.
- Keep JavaScript in external files and load it with defer or at a safe point in the document.

JavaScript concept ladder

Track	Concepts you should be able to explain
Grade 3	DOM selection, addEventListener(), preventDefault(), values/textContent, classList, conditions, strings and setTimeout().
Grade 4 adds	Arrays, objects, loops/forEach(), querySelectorAll(), dataset, filter(), sort() and template literals or DOM creation.
Grade 5 adds	DOMContentLoaded, state classes, localStorage or URLSearchParams, accessible pressed/active state and early settings application.

Suggested file structure

```
portfolio-website/
├── index.html
├── feedback.html          # Grade 3+
├── skills.html            # Grade 4+
├── css/
│   ├── style.css
│   ├── feedback.css       # Grade 3+
│   ├── skills.css         # Grade 4+
│   └── a11y.css           # Grade 5
├── js/
│   ├── feedback.js        # Grade 3+
│   ├── skills.js          # Grade 4+
│   └── a11y.js            # Grade 5
├── images/
├── .gitignore
└── README.md
```

File names may differ

You may integrate a feature into an existing page or use another clear folder structure. The important point is that files are easy to find, links work and the README explains how to open the project.

SECTION 09

Seminar & code defense

Plan for approximately five minutes unless the published seminar schedule says otherwise. The goal is to demonstrate working behavior and explain the code behind it.

Before your turn

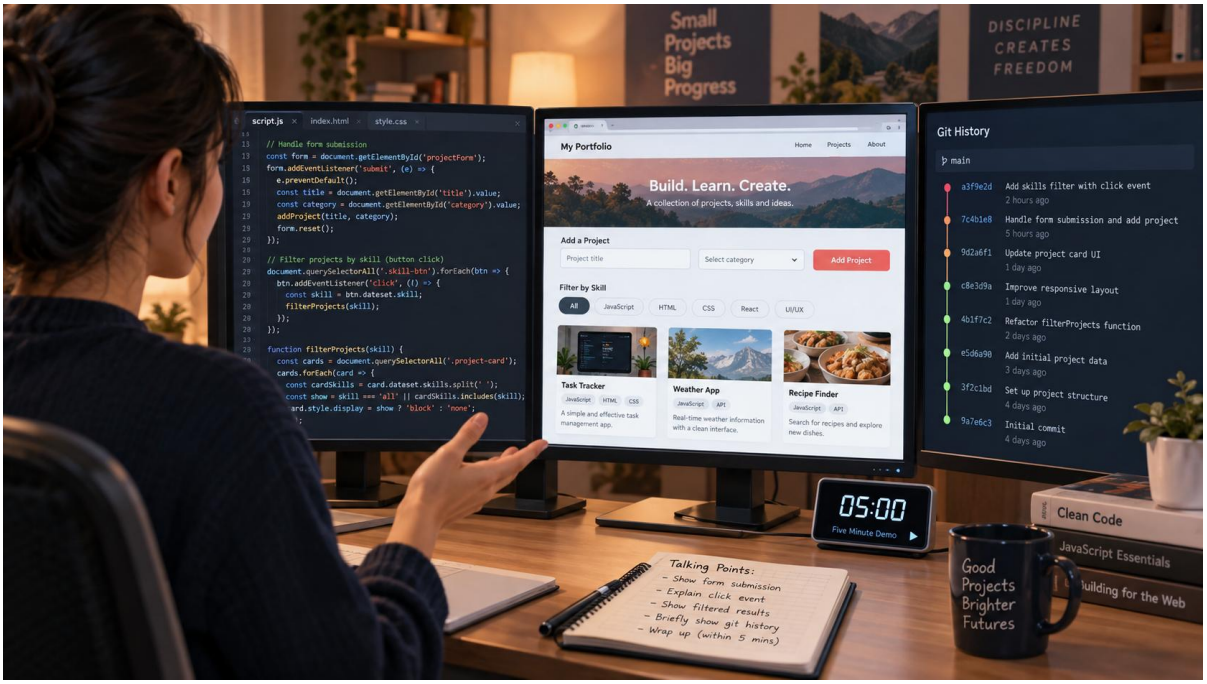
- Open the site in a browser, the relevant JavaScript files in your editor and the repository commit history.
- Confirm that the browser console is clear and prepare realistic test input.
- Know which feature set your target grade requires and where the relevant functions are located.

Live demonstration

- Demonstrate the required feature or features for your target grade.
- Trace one interaction from the event, through the JavaScript logic, to the visible DOM update.
- Show a short section of Git history and connect one commit to a decision or fix.
- Answer short questions about validation, arrays/filtering/sorting, persistence or accessibility, depending on your track.

If something is unclear

I may ask a follow-up question or ask you to locate a related part of the code before grading is completed. A simple, accurate explanation is stronger than a memorized definition.



Prepare the browser, code and commit history so the demonstration stays focused and calm.

SECTION 10

FAQ & resources

Short answers to the questions most likely to affect your implementation or submission.

Can I use Bootstrap or another CSS framework?

Yes, for styling. The JavaScript must still be written in vanilla JavaScript without framework-specific behavior.

Can I use AI while debugging?

You may use it as a debugging aid within the course rules, but you are responsible for checking the solution and explaining every submitted change during the seminar.

What if the higher-track feature is not working?

Submit the highest complete, reliable track you can demonstrate. Clearly describe any unfinished experiment rather than presenting it as complete.

Can classmates work together?

You may discuss general approaches, but write and test your own code. Substantially matching solutions may require individual clarification before grading.

How detailed should commit messages be?

Describe the change clearly: “Add email validation” is useful; “update” is not.

Must Grade 5 use localStorage?

No. URL parameters are an accepted alternative when they restore the selected view after reload and you can explain the implementation.

Recommended references

- MDN JavaScript Guide
- MDN Client-side form validation
- MDN Web Storage API
- MDN URLSearchParams
- Pro Git · Getting Started

Debugging tip

Use the browser developer tools to inspect the console, event behavior, element classes and responsive layout. Fix the first relevant error before chasing later symptoms.

SECTION 11

Submission checklist

Complete the common checks and every check under your cumulative target track.

All grades

- ☐ The repository link works and the complete commit history is visible.
- ☐ The minimum total is met for my target grade: 10, 25 or 45 meaningful commits.
- ☐ .gitignore and README.md are included; the README states my target grade and how to run the site.
- ☐ The short report includes project information, required screenshots, code explanation, problem solving and reflection.
- ☐ The website loads without missing files or browser-console errors.
- ☐ All features work at mobile and desktop widths and remain keyboard accessible.
- ☐ I can open the relevant code and Git history quickly for the seminar.

Grade 3 feature

- ☐ Every required field is validated and the optional URL uses browser validation when filled.
- ☐ Errors appear beside the correct field and clear when the value becomes valid.
- ☐ The character counter updates live and communicates the 15-character minimum in text and color.
- ☐ A successful submission shows the personalized message and resets the full form state.
- ☐ The success message disappears after about 4 seconds, and Clear restores the initial state.

Higher tracks continue on the next page

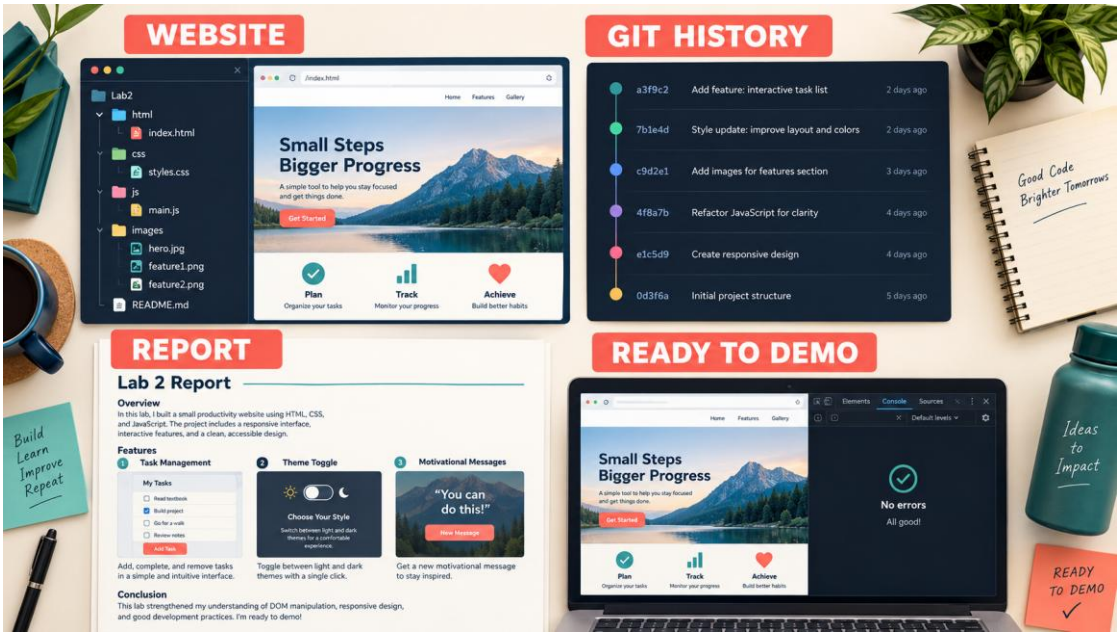
Grade 4 students complete the Grade 3 checklist plus Grade 4. Grade 5 students complete all three feature checklists.

Grade 4 addition

- ☐ The array contains at least 8 skills across at least 3 categories with all required properties.
- ☐ Skill cards are rendered from the array when the page loads.
- ☐ Every category filter works, the active filter is clear and the result counter updates.
- ☐ All three sort choices work and sorting remains active together with the selected filter.
- ☐ The controls work with the keyboard and the interface does not rely on color alone.

Grade 5 addition

- ☐ Small, Medium and Large text settings work without breaking the layout.
- ☐ High Contrast remains readable across navigation, cards, forms, messages and focus states.
- ☐ All panel controls work with Tab and Enter or Space and expose their active/pressed state.
- ☐ The selected settings return after reload using localStorage or URL parameters.
- ☐ All six size-and-contrast combinations were tested without console errors.



Final submission package: working website, traceable process, concise report and a prepared live demonstration.

Final reminder

The illustrations in this brief explain the expected behavior. Your portfolio, data, code, screenshots, commit history and reflection must represent your own work.